Namma Space: Smartphone Video to a Walkable Photorealistic Indoor Twin

Team ID: TM-12354C3F6F2 · **Institute:** Vellore Institute of Technology · **Round 1 Technical Abstract**, Techfest 2026-27, IIT Bombay
Members: Nilesh Kumar Srivastava, Ojas Ashish Kulkarni, Ujesha Sivaramakrishnan, **[ASHWIN_FULL_NAME]**
All links (live viewer, repository, dataset, demo video) are in *Additional_Links_and_Prototypes.pdf*.

Summary. One person walks a room with an ordinary phone, following a short capture SOP. A Python pipeline turns the video into a 3D Gaussian Splatting (3DGS) model. ffmpeg samples frames and drops blurred ones; COLMAP recovers camera poses; Brush trains the splat on any WebGPU GPU; splat-transform compresses it to a ≤ 50 MB web file. The result is served from a CDN to a zero-install browser viewer (Spark on three.js) with first-person WASD + mouse controls and a touch fallback. Every stage has an automated quality gate, and a run counts as successful only after a person has walked the result and signed it off.

1. Problem and scope

Floor plans do not tell a visitor what a space looks like, and photo tours cannot be walked. **Round 1** targets the core loop: **capture → photorealistic twin → free roam in the browser**, reproducible by a non-expert with one phone and one consumer NVIDIA GPU. Fidelity comes from the capture SOP and the pipeline's gates rather than manual modelling. **Rounds 2–3:** the repository already contains a Round 2 studio app (`apps/studio`) with a validated scene-bundle format, POI search and A* routing over an occupancy grid. Round 2 will attach those to the Round 1 splat in a shared metre-scaled frame. Round 3 adds on-site capture via phone pairing (prototype in the studio) and optional AR navigation.

2. System architecture

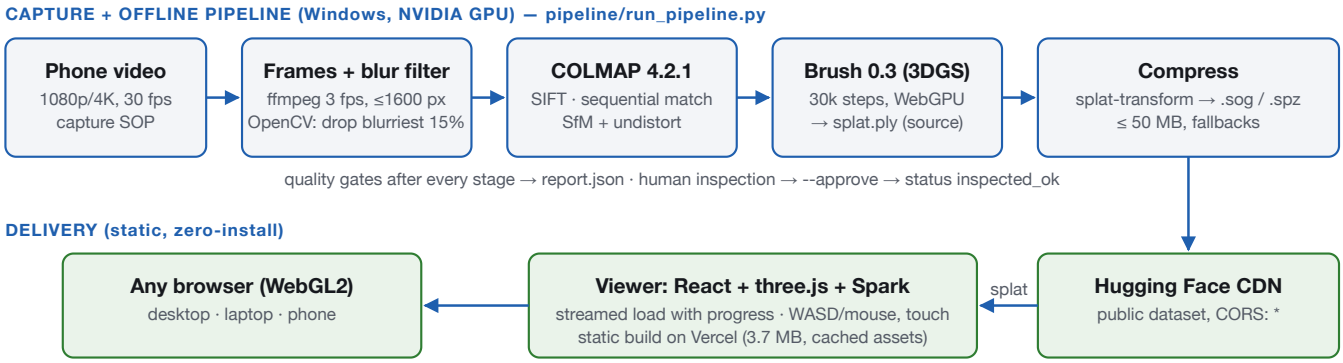


Figure 1. End-to-end architecture. Heavy work runs offline on one GPU machine; the web side is two static origins (app on Vercel, splat on a CORS-enabled CDN) with no server code.

The pipeline is a single Python CLI (`pipeline/run_pipeline.py`) with resumable stages. Each stage writes its outputs and check results to `work/<name>/report.json`, the one source of truth for a run's state (`failed` → `automated_checks_passed_awaiting_inspection` → `inspected_ok`). The viewer reads a small `scene.json` (splat URL, spawn, scale, rotation, optional walk bounds), so a new capture needs no code change.

3. Capture SOP: rules and why they exist

Rule	Why
Lock exposure and focus (long-press AE/AF); 1x lens, no zoom; stabilisation and HDR off; no setting changes mid-clip	Structure-from-motion assumes one camera with fixed intrinsics. Zoom, digital stabilisation and focus hunting change the effective camera from frame to frame and warp straight walls. Exposure pumping changes pixel brightness, which hurts feature matching and bakes flicker into the splat's colours.
Pace: slow walk ~0.5 m/s, pan ≤ 20°/s, never spin in place	Motion blur is the top cause of failed reconstructions. At 3 fps sampling and 0.5 m/s, consecutive frames are ~17 cm apart: dense overlap with little blur.

Overlap: perimeter loop facing ~20° into the wall, two centre passes, corner pans, half-arcs round furniture; cross doorways slowly both ways; **end where you started**	Every surface must appear in 3+ frames from different positions for triangulation and view-dependent colour. Closing the loop lets bundle adjustment cancel drift. Doorway crossings in both directions stop rooms splitting into disconnected models.
Lighting: all lights on, curtains open, no direct sun beams, no flash	Even, constant illumination gives consistent colours across views. Blown highlights and moving flash shadows become baked-in artefacts.
Avoid mirrors, glass, windows, plain white walls, moving people	Reflections and transparency violate the static-scene assumption and produce "floaters". Texture-less walls give SIFT nothing to match, so keep edges, switches or posters in frame. Moving people break the static-scene assumption.
Length: ≥ 31 s (aim 60–120 s per room, ≤ 900 s); landscape 1080p/4K at 30 fps	The pipeline needs ≥ 80 usable frames after dropping the blurriest 15 % at 3 fps: ⌊31·3⌋ = 93, minus 13 dropped = 80. `--check` computes this from the clip's real duration and rejects shorter clips *before* any processing, naming the minimum length.

A 60-second on-phone checklist (scrub for blur and dark sections, doorways covered both ways, loop closed, no lingering people) catches re-shoots before leaving the site. The SOP also documents failure modes and fixes, and how to set metric scale from one known length (e.g. a 0.9 m doorway).

4. Pipeline design and quality gates

Stage	What and key parameters	Automated gate (blocks the run unless noted)
Validate	ffprobe resolution, duration, fps; free disk; dependency doctor (COLMAP, Brush, ffmpeg, Node ≥ 22, splat-transform, OpenCV)	short side ≥ 720 px; length gives ≥ 80 usable frames at the chosen fps; ≤ 900 s; ≥ 10 GB free. Warnings: < 30 s, < 24 fps. Photo input: ≥ 40 images, any extension case, HEIC flagged.
Frames	ffmpeg at 3 fps, longest side ≤ 1600 px, JPEG q2; OpenCV variance-of-Laplacian ranks sharpness and drops the blurriest 15 %	≥ 80 usable frames (same formula as Validate, so a clip that passes `--check` cannot fail here)
SfM	COLMAP 4.2.1 (CUDA): SIFT extraction, single shared OPENCV camera; sequential matching for video (exhaustive for photo sets); incremental mapper; best of any sub-models; `image_undistorter`	model exists; registered ≥ 60 % of frames, ≥ 2 000 sparse points. Warnings: registration < 90 %, reprojection error > 1.5 px, room split into several models. **Acceptance bar: ≥ 0.90 registration**, else re-shoot.
Train	Brush 0.3.0, 3DGS, 30 000 steps, headless CLI; single export at the last step. Runs on any WebGPU GPU with no CUDA toolkit (Nerfstudio splatfacto as an alternative trainer).	**Stale-splat guard:** a splat is reused only if trainer, steps and COLMAP model fingerprint all match; otherwise the export folder is wiped first. Dataset guard: exactly one COLMAP model, images ≥ registered (Brush silently skips missing ones), no stray `.ply` (Brush would use it as its starting point).
Splat checks	parse the binary PLY header and memory-map the data	all 14 3DGS attributes present; ≥ 50 000 splats; byte count matches header (not truncated); positions and opacity finite; non-degenerate extent
Compress	splat-transform 3.10.3: NaN filter, then `.sog` (~10x smaller). Fallbacks: GPU → CPU encoding → `.spz` v3.	**Size-targeted:** ≤ 50 MB (`--max-mb`). Over budget: drop spherical-harmonic bands 3 → 2 → 1 → 0, then decimate uniformly to ~10 % under budget (≤ 3 passes); output magic bytes verified.
Publish + inspect	`scene.json` + compressed splat to `work/<name>/web/` and the dev server; the raw `.ply` stays as the source of truth and never enters the web build	**Human gate:** walk the whole room; `--approve --note` sets `inspected_ok`. Bar: ≥ 0.90 registration, no floaters or smeared walls on the walk path, doorways and furniture recognisable.

Stages are skipped when their verified outputs exist, so a failed run resumes where it stopped (`--force` restarts). `--check` validates without touching a run's status. The checks only prove outputs are well-formed (a synthetic file passes them too), which is why success is declared only by the human gate.

5. Web viewer

- **Rendering.** React 19 + react-three-fiber 9 over three.js 0.186, with Spark 2.4 rendering Gaussian splats inside a normal three.js scene. Later POI markers, paths and collision helpers are ordinary scene objects. The pixel ratio is capped at 1.5 and MSAA is off, to hold frame rate on laptops and phones.
- **Controls.** Desktop: click "Enter space" to capture the pointer; mouse look, `W A S D`/arrows to walk (1.6 m/s), `Shift` to sprint (3.4 m/s), `Q` / `E` to lower or raise the eye, `Esc` to release. Movement stays on the floor plane whatever the pitch, with optional axis-aligned walk bounds. Touch: drag to look, hold two fingers to walk forward.
- **Progressive loading and errors.** The splat is streamed with `fetch` and a byte-accurate progress bar ("x / y MB", then "Preparing the splat…" while decoding). Failures become plain messages: 404, private or missing file (401/403), an HTML page instead of a splat, a CORS/network block, or a file whose magic bytes are not PLY/SPZ/SOG.
- **Hosting.** The app is a static Vite build on Vercel: 3.7 MB, a single JS bundle of 1.27 MB gzip, hashed assets cached as immutable. The splat lives on a public Hugging Face dataset; requests are redirected to its CDN, which returns `Access-Control-Allow-Origin: *`. App and data deploy independently, and test fixtures are served only by the dev server, never shipped.
- **Why splats over meshes.** 3DGS reproduces view-dependent appearance (gloss, soft lighting) and thin or cluttered geometry that photogrammetry meshes blur or break. It needs no dense multi-view stereo, meshing or texture baking. It trains in tens of minutes on one consumer GPU and renders in real time in WebGL2. Its weakness, having no clean surface for collision, is handled by a separate navigation layer (Section 7).

6. Results

Values from the real-capture run (`report.json`) and measurements of the deployed viewer.

Metric	Value	Metric	Value
Clip length / frames used	**[FRAMES]**	Splat count (Brush)	**[SPLAT_COUNT]**
COLMAP registration ratio	**[REG_RATIO]**	Web file size (format)	**[FILE_MB]**
Stage timings (frames / SfM / train / compress)	**[STAGE_TIMINGS]**	Frame rate (device)	**[FPS]**
Load time, desktop (broadband)	**[LOAD_TIME_DESKTOP]**	Load time, phone (4G/Wi-Fi)	**[LOAD_TIME_PHONE]**

[SCREENSHOT_1]

Figure 2. **[SCREENSHOT_1_CAPTION]**

[SCREENSHOT_2]

Figure 3. **[SCREENSHOT_2_CAPTION]**

Compression on the synthetic test room (105,991 splats, 6.9 MB PLY): `.spz` 0.86 MB (8x), `.sog` 0.69 MB (10x), each rendering the same room. These ratios are illustrative, not a prediction for a real capture.

7. Limitations and next steps

- **No object collision yet.** Round 1 clamps movement to an optional rectangle, so walls and furniture can be walked through. Round 2 adds a metre-scaled occupancy grid (prototyped in `apps/studio`), with floor-plane fitting on COLMAP points to replace manual alignment.
- **POIs, search and routing already exist** in `apps/studio`: a zod-validated scene bundle (navigation grid, POIs, quality report), POI search and A* routes, with unreachable targets reported honestly. Round 2 connects them to the Round 1 splat.

- **Reconstruction limits.** Mirrors, glass and blank walls remain hard, and scale and orientation are set by hand from one known length. Compression is lossy; each real splat is compared against its raw `.ply` before shipping.
- **Round 3.** On-site capture through phone pairing (WebRTC pairing and frame-transfer prototypes exist in the studio) and optional AR navigation against the twin.

8. Open-source credits

Tool	Use in Namma Space	Licence
COLMAP 4.2.1	SIFT features, matching, structure-from-motion, undistortion	BSD-3-Clause
Brush 0.3.0	3D Gaussian Splatting training (WebGPU)	Apache-2.0
Nerfstudio (optional)	alternative splat trainer (splatfacto)	Apache-2.0
FFmpeg / ffprobe	frame sampling, video probing	LGPL-2.1+ (GPL builds: GPL-2.0+)
OpenCV (opencv-python-headless)	blur scoring (variance of Laplacian), image checks	Apache-2.0
NumPy	splat PLY validation	BSD-3-Clause
@playcanvas/splat-transform 3.10.3	PLY → SOG/SPZ compression, SH reduction, decimation	MIT
Spark (@sparkjsdev/spark) 2.4	Gaussian splat rendering in three.js	MIT
three.js 0.186	WebGL scene and camera	MIT
react-three-fiber 9	React bindings for three.js	MIT
React 19, React DOM	viewer and studio UI	MIT
Vite, TypeScript, oxlint, ESLint	build, type checking, linting	MIT; TypeScript Apache-2.0
Node.js	build tooling, runs splat-transform	MIT
PlayCanvas engine (studio)	Round 2 viewer prototype	MIT
zod, fflate, idb, ws, qrcode-generator (studio)	schemas, ZIP export, IndexedDB, dev signalling relay, QR pairing	MIT; idb ISC
Vitest, Playwright (studio)	unit and browser tests	MIT; Playwright Apache-2.0
Archivo, IBM Plex Mono fonts (studio)	UI typography	SIL OFL-1.1
GSAP (studio)	UI motion	GSAP Standard "no charge" licence (not OSI)

Hosting services (not open source): Vercel (static app), Hugging Face Hub (splat storage and CDN). GLOMAP is not used; COLMAP's incremental mapper is the only SfM backend.